

PAGE REPLACEMENT ALGORITHM SIMULATOR

Interactive Web-Based Visualizer for FIFO, LRU, LFU & Optimal Page Replacement Algorithms

This is Submitted By:

Ayush Sharma (12406797)

Harshita Sawhney(12410095)

Arzoo (12407625)

Section-9S394

Under the Guidance of

Prerita Agarwal



LOVELY
PROFESSIONAL
UNIVERSITY

School of Computing & Artificial Intelligence
Lovely Professional University, Phagwara

DECLARATION

We hereby certify that the project titled "Page Replacement Algorithm Simulator" is a genuine representation of our work, conducted as part of the Project requirement for the CA-CSE275 at Lovely Professional University, Phagwara. This work was carried out under the supervision of Professor Prerita Agarwal during the period of January to April 2026. All the data and insights presented in this project report are derived from our dedicated efforts and are entirely original.

CERTIFICATE

This is to certify that the declaration statement made by this group of students is correct to the best of my knowledge and belief. They have completed this Project under my guidance and supervision. The present work is the result of their original investigation, effort and study. No part of the work has ever been submitted for any other degree at any University. The Project is fit for the submission in Lovely Professional University, Phagwara.

Prerita Agarwal

Designation: Assistant Professor

School of Computing & Artificial Intelligence

Lovely Professional University,

Phagwara, Punjab

Date: 25-04-2026

ABSTRACT

Virtual memory management is a cornerstone of modern operating systems, and the algorithm used to decide which page to evict when a page fault occurs directly determines system performance. This project presents the design and development of a Page Replacement Algorithm Simulator, an interactive, browser-based educational tool that implements and visually compares four major page replacement strategies: First-In-First-Out (FIFO), Least Recently Used (LRU), Least Frequently Used (LFU), and Belady's Optimal (OPT) algorithm. Built entirely with HTML5, CSS3, and vanilla JavaScript (ES2022), and enhanced with Chart.js for data visualization, Three.js for an animated background, and html2canvas for exporting results, the application allows a user to enter any page reference string, select the number of physical memory frames, and observe step-by-step how each algorithm loads, retains, and evicts pages. The simulator computes and displays key performance metrics — page faults, hits, hit ratio, and fault rate — for all four algorithms simultaneously, alongside comparative bar and line charts. A dedicated module reproduces Belady's Anomaly, the counter-intuitive case in which FIFO produces more faults as the number of available frames increases. The system was verified against classical reference strings from Silberschatz's Operating System Concepts, with computed results matching the expected textbook fault counts exactly. Deployed as a live, publicly accessible web application on GitHub Pages, the project demonstrates that complex, interactive algorithm-visualization tools can be built using only standard client-side web technologies, offering students and educators an intuitive, hands-on way to understand virtual memory management.

TABLE OF CONTENT

[illegible]

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

In a modern operating system, virtual memory allows a process to use more memory than is physically available by keeping only a subset of its pages resident in RAM at any given time. When a process references a page that is not currently in physical memory, a page fault occurs, and the operating system must decide which resident page to evict to make room for the incoming page. The policy governing this eviction decision is known as the page replacement algorithm, and it has a direct, measurable effect on system performance. A poorly chosen policy can lead to thrashing, a pathological state in which the CPU spends more time servicing page faults than executing useful process instructions.

Several page replacement strategies exist, including First-In-First-Out (FIFO), Least Recently Used (LRU), Least Frequently Used (LFU), and Belady's theoretically optimal algorithm (OPT). Although these algorithms are described extensively in operating system textbooks, students often find it difficult to build an intuitive understanding of how they behave, how their fault counts differ for the same reference string, and why certain algorithms outperform others under specific access patterns. Traditional classroom instruction relies largely on manual, static trace tables worked out on paper, which are time-consuming to construct, error-prone, and difficult to use when exploring multiple scenarios quickly. There is a clear need for an accessible, interactive tool that allows a user to experiment with different reference strings and frame counts and immediately observe and compare the behaviour of all major page replacement algorithms.

1.2 Motivation

Efficient memory management is one of the most performance-critical responsibilities of an operating system, and the choice of page replacement policy affects everything from application responsiveness to overall system throughput. For students and early-career developers, the gap between reading about an algorithm in a textbook and truly understanding its runtime behaviour can be significant. Static, paper-based worked examples convey the mechanics of a single algorithm on a single input but rarely provide the opportunity to directly compare multiple algorithms side by side, experiment with different reference strings, or witness counter-intuitive behaviours such as Belady's Anomaly, in which increasing the number of available frames unexpectedly increases the number of page faults produced by FIFO.

An interactive, visual simulator addresses this gap directly. By allowing a user to enter any reference string, choose a frame count, and instantly see the frame contents, hit and fault outcomes, and aggregate statistics for FIFO, LRU, LFU, and OPT simultaneously, abstract theoretical concepts become tangible and immediately verifiable. Such a tool also has value

beyond the classroom: it can serve as a quick reference for checking manually computed results, as a demonstration aid for instructors, and as a portfolio project that shows the ability to translate operating-system theory into a fully functional, deployed software application using only client-side web technologies. This motivated the development of a fully client-side, dependency-light simulator that runs in any modern browser without installation.

1.3 Objectives of the Study

The primary objective of this project is to design, implement, and deploy an interactive web-based simulator that helps students and educators understand and compare page replacement algorithms used in operating system memory management.

The first objective is to correctly implement the four major page replacement algorithms — FIFO, LRU, LFU, and OPT — as independent, verifiable computational modules that accept any valid page reference string and frame count and return a complete, step-by-step trace along with aggregate statistics.

The second objective is to design a clear and intuitive visual representation of each algorithm's execution, including a colour-coded frame-state table that distinguishes hits, faults, newly loaded pages, and evicted pages, so that the internal working of each algorithm is transparent to the user at every step.

The third objective is to provide quantitative performance comparison across all four algorithms through computed metrics such as total faults, total hits, hit ratio, fault rate, and a simulated miss penalty, supported by bar and line charts that make relative performance immediately visible.

The fourth objective is to reproduce and clearly demonstrate Belady's Anomaly, a classical counter-example that illustrates why FIFO is not a stack algorithm, thereby reinforcing an important theoretical concept through direct, hands-on observation.

Finally, the study aims to deploy the simulator as a live, publicly accessible web application, verify its correctness against established textbook reference strings, and identify avenues for future enhancement such as additional algorithms, multi-process simulation, and backend persistence.

CHAPTER 2

LITERATURE REVIEW

2.1 Related Work

The theoretical foundation for page replacement algorithms was established by Belady, whose 1966 study of replacement strategies for virtual-storage computers introduced the concept of an optimal replacement policy and first identified the anomaly, now known as Belady's Anomaly, in which certain non-stack algorithms such as FIFO can produce more page faults when allocated additional memory frames. King later provided one of the earliest systematic comparative analyses of paging algorithms, examining the trade-offs between simplicity and fault-rate performance across competing strategies. These foundational works remain the primary theoretical reference for FIFO, LRU, LFU, and OPT, and are reflected in standard operating-system textbooks such as Silberschatz, Galvin and Gagne's *Operating System Concepts* and Tanenbaum and Bos's *Modern Operating Systems*, both of which this project draws upon for algorithm definitions and correctness benchmarks.

Beyond the algorithmic theory, a substantial body of computer-science education research has explored the use of interactive visualization tools to teach operating-system concepts, motivated by the observation that memory management is notoriously difficult for students to grasp from static diagrams alone. Khuri and Hsu presented an early interactive package for visualizing the behaviour of the CPU scheduler and several page replacement algorithms, reporting that self-paced, animated simulation improved student engagement compared with lecture-only instruction. Sibai and colleagues extended the Java-based MOSS virtual memory simulator to let students switch freely between FIFO, LRU, and Optimal replacement policies and directly compare page-fault statistics, an approach that closely parallels the simultaneous multi-algorithm comparison implemented in this project. Null and Rao's CAMERA tool took a similar visualization-first approach specifically for introducing memory-management concepts to undergraduates.

More recent work has moved this style of tool onto the web. Garmpis described Alg_OS, a browser-based application that allows students to explore performance metrics for Optimal, FIFO, LRU, and Clock algorithms without requiring any local software installation, while Wani presented a virtualization and visualization tool aimed at making the abstract technical details of the virtual memory system more approachable for learners. Across this literature, a consistent finding is that algorithm-visualization technology is most effective when it lets learners predict outcomes and immediately check them against the simulated result, rather than passively observing an animation — a principle that directly informed the step-through mode and instant multi-algorithm comparison built into the present simulator.

Taken together, this body of work confirms two things that shaped the present project: first, that FIFO, LRU, LFU, and OPT remain the standard set of algorithms used to teach page replacement, with well-established textbook reference strings available for correctness verification; and second, that browser-based, dependency-light visualization tools are an established and effective way to make these algorithms tangible for students, without requiring specialised software installation. This project builds on that foundation by implementing all four algorithms as verifiable, side-by-side comparisons within a single modern web application, extending the educational approach of earlier Java-based tools with contemporary web technologies such as Chart.js for charting and html2canvas for exportable results.

CHAPTER 3

METHODOLOGY

3.1 System Architecture and Module Design

The Page Replacement Algorithm Simulator is architected as a single-file, client-side web application built entirely with HTML5, CSS3, and JavaScript (ES2022), requiring no backend server, database, or compilation step. This design choice keeps the application lightweight, instantly deployable on any static hosting platform, and universally accessible without installation.

The system is organised into three loosely coupled modules. The Algorithm Engine forms the computational core and consists of four pure functions — `simulateFIFO()`, `simulateLRU()`, `simulateLFU()`, and `simulateOPT()` — each of which accepts a page reference string and a frame count and returns a detailed, step-by-step execution trace together with aggregate statistics such as total faults and hits. Because these functions have no side effects and do not interact with the Document Object Model (DOM), they can be tested and verified independently of the user interface.

The Visualization and UI Layer transforms the raw output of the algorithm engine into an interactive visual experience. It renders a colour-coded frame-state table for each algorithm, animated statistics cards showing hit ratio and fault rate, a “winner” card that announces the best-performing algorithm for a given input, and a step-through mode that allows the user to advance through the simulation one reference at a time using keyboard navigation.

The Chart and Export Module handles graphical data representation and result export. It uses the Chart.js library to render a comparative bar chart of total page faults and a cumulative line chart of faults over time for all four algorithms simultaneously, and integrates `html2canvas` so that the results panel can be exported as a PNG image, along with a structured CSV export of the full simulation trace.

3.2 Algorithm Design and Implementation

Each of the four page replacement algorithms is implemented according to its standard theoretical definition. The FIFO algorithm maintains a circular queue of resident pages; when a fault occurs and memory is full, the page that has been resident the longest is evicted, giving a time complexity of $O(n \cdot m)$, where n is the length of the reference string and m is the number of frames. The LRU algorithm maintains a timestamp map recording the last access index of every resident page and evicts the page with the oldest timestamp on a fault; because LRU is a stack algorithm, it is immune to Belady's Anomaly. The LFU algorithm maintains a frequency counter and a load-order map for each resident page, evicting the page with the lowest access frequency and breaking ties by evicting the page that was loaded earliest. The OPT algorithm, used as a theoretical

benchmark, scans forward in the reference string to determine each resident page's next future use and evicts the page whose next use lies farthest in the future; it is provably optimal but cannot be implemented in a real operating system because it requires advance knowledge of future references.

3.3 Development and Testing Methodology

The application was developed incrementally using Git version control, with a dedicated feature branch created for each major module — including feature/fifo-algo, feature/lru-lfu, feature/optimal, feature/charts, and feature/animations — before being merged into the main branch. This branch-per-feature strategy produced a clean, demonstrable revision history of more than seven commits and allowed each algorithm implementation to be tested in isolation before integration.

Correctness was verified by running each algorithm against classical reference strings drawn from Silberschatz, Galvin and Gagne's Operating System Concepts and comparing the computed fault counts against the expected textbook results. Development was carried out in Visual Studio Code, with Chrome DevTools used for browser-side debugging, responsive-design testing, and performance profiling. The completed application was deployed on GitHub Pages, providing a live, publicly accessible URL requiring no installation.

3.4 Evaluation Metrics

The performance of each algorithm is evaluated using standard memory-management metrics computed directly from the simulation trace: total page faults, total hits, hit ratio (the percentage of references satisfied without a fault), fault rate (the percentage of references that caused a fault), and a simulated miss penalty calculated as the number of faults multiplied by a fixed per-fault cost, used as an educational approximation of disk-access overhead. The algorithm producing the fewest faults for a given input is automatically flagged as the best performer for that run. In addition, a dedicated diagnostic module reproduces Belady's Anomaly by running FIFO with three and four frames on the classical anomaly reference string (1 2 3 4 1 2 5 1 2 3 4 5) and automatically comparing the resulting fault counts.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Experimental Discussion

The simulator was evaluated using the classical reference string from Silberschatz's Operating System Concepts — 7 0 1 2 0 3 0 4 2 3 0 3 2 — with three physical memory frames. Running all four algorithms simultaneously on this input produced the following fault counts: FIFO recorded 9 faults, LRU recorded 8 faults, LFU recorded 8 faults, and OPT recorded 7 faults. These results match the expected textbook outcomes exactly, confirming that each algorithm was implemented correctly. As expected, the Optimal algorithm produced the fewest faults of the four, since it has provably minimal fault behaviour, while FIFO produced the most, consistent with its lack of awareness of page usage recency or frequency.

4.2 Testing

Beyond the Silberschatz benchmark, the simulator was tested against a range of additional reference strings, including sequential access patterns, looped access patterns, a thrashing scenario, and the classical Belady's Anomaly string. In every case, the frame-state trace produced by each algorithm was manually cross-checked against hand-computed traces to confirm correctness. The dedicated Belady's Anomaly module correctly reproduced the anomaly: running FIFO with three frames on the reference string 1 2 3 4 1 2 5 1 2 3 4 5 produced fewer faults than running the same algorithm with four frames, confirming the counter-intuitive result that adding more physical memory can, under FIFO, increase the number of page faults. LRU, LFU, and OPT did not exhibit this anomaly on the same input, consistent with theoretical expectations.

4.3 Performance Insights

Across all tested reference strings, OPT consistently produced the lowest number of page faults, serving as a useful theoretical lower bound against which the practical algorithms can be measured. LRU and LFU produced results very close to OPT for reference strings with strong temporal or frequency locality, confirming that these algorithms approximate optimal behaviour reasonably well in practice despite requiring no knowledge of future references. FIFO consistently produced the highest number of faults among the three practically implementable algorithms and was the only algorithm to exhibit Belady's Anomaly, reinforcing the theoretical distinction between stack algorithms such as LRU and OPT and non-stack algorithms such as FIFO.

Table 1: Result Comparison on the Silberschatz Reference String (3 Frames)

--	--	--	--

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Page Replacement Algorithm Simulator successfully delivers on all objectives set out for this project. All four major operating-system page replacement algorithms — FIFO, LRU, LFU, and Belady's Optimal — have been implemented with correct, verifiable logic, tested against known reference strings including classical textbook examples from Silberschatz's Operating System Concepts, and wrapped in a visually rich, highly interactive web application.

The comparative analysis enabled by the simulator makes the theoretical differences between algorithms immediately tangible. For the classic Silberschatz reference string with three frames, FIFO produced 9 faults, LRU and LFU each produced 8 faults, and OPT produced 7 faults, exactly matching textbook results and confirming algorithm correctness. Belady's Anomaly was reliably reproduced and clearly demonstrated through a dedicated interactive module.

From a technical standpoint, the use of pure vanilla JavaScript, without any front-end framework, demonstrates that complex, interactive data-visualization applications can be built with standard web technologies alone. The application is fully responsive, supports keyboard navigation, and offers both light and dark themes. Deployed live on GitHub Pages with a documented, feature-branch-based revision history, the project stands as a complete, working demonstration of how operating-system theory can be translated into an accessible, browser-based educational tool.

5.2 Future Work

Although the outcomes of this project are encouraging, several directions remain open for future improvement and extension:

- Clock (Second-Chance) Algorithm: implement the clock replacement algorithm, which approximates LRU using a circular buffer and reference bits and is the algorithm actually used by Linux.

- Working Set Model: add simulation of the working set model and working set window, demonstrating how an operating system tracks the set of pages actively used by a process.

- Multi-Process Simulation: extend the simulator to model multiple concurrent processes with their own page tables competing for shared physical frames, introducing global versus local replacement policies.

- Backend Integration: add a lightweight backend to persist simulation sessions, allowing users to share simulation URLs and compare results asynchronously.

Batch and CSV Import: allow users to upload a file of page reference strings for batch simulation, enabling analysis of real-world memory access traces.

Automated PDF Reporting: generate a complete PDF report directly from simulation results, including charts, tables, and analysis text, ready for academic submission.

Performance Profiling: add timing measurements for each algorithm's execution to compare computation time in addition to fault counts.

Accessibility Enhancements: add full screen-reader support, high-contrast mode, and improved keyboard-only navigation.

CHAPTER 6

REFERENCES

- [1] Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 10th Edition. Wiley, 2018. ISBN: 978-1-119-32091-3. [Primary theoretical reference for page replacement algorithm descriptions, Belady's Anomaly, and virtual memory concepts.]
- [2] Andrew S. Tanenbaum, Herbert Bos. Modern Operating Systems, 4th Edition. Pearson Prentice Hall, 2014. ISBN: 978-0-13-359162-0. [Secondary reference for LRU clock approximation, working sets, and page fault handling.]
- [3] L. A. Belady. "A study of replacement algorithms for a virtual-storage computer." IBM Systems Journal, vol. 5, no. 2, pp. 78–101, 1966. [Original paper introducing the Optimal algorithm and Belady's Anomaly for FIFO.]
- [4] W. F. King. "Analysis of paging algorithms." Proceedings of the IFIP Congress, pp. 485–490, 1971. [Early comparative analysis of page replacement strategies.]
- [5] S. Khuri and H. C. Hsu. "Visualizing the CPU scheduler and page replacement algorithms." Proceedings of the SIGCSE 1999 Technical Symposium on Computer Science Education, pp. 227–231, 1999.
- [6] M. A. Sibai et al. "Teaching page replacement algorithms with a Java-based VM simulator." [Java-based MOSS virtual memory simulator supporting FIFO, LRU and Optimal replacement comparison.]
- [7] L. Null and K. Rao. "CAMERA: introducing memory concepts via visualization." Proceedings of the 36th SIGCSE Technical Symposium on Computer Science Education, pp. 96–100, 2005.
- [8] A. Garmpis. "Alg_OS: A web-based software tool to teach page replacement algorithms of operating systems to undergraduate students."
- [9] F. F. Wani. "Virtualization and visualization of virtual memory system for effective teaching-learning."
- [10] Chart.js Documentation, v4.4.0. <https://www.chartjs.org/docs/latest/> [Bar and line chart implementation, animation and tooltip configuration.]
- [11] Three.js Documentation, r128. <https://threejs.org/docs/> [WebGL renderer setup, particle geometry, animation loop.]
- [12] html2canvas Documentation, v1.4.1. Niklas von Hertzen. <https://html2canvas.hertzen.com/> [DOM-to-canvas capture API used for PNG export.]
- [13] GitHub Docs — GitHub Pages. <https://docs.github.com/en/pages> [Static site deployment and branch publishing.]
- [14] MDN Web Docs — JavaScript Reference. Mozilla Foundation. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>